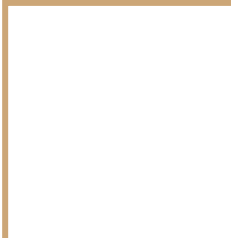


Data Structures

Mushtari Sadia





Recursion



Definition

- A programming technique where a **function calls itself**
 - in programming, goal is to NOT have infinite recursion
 - must have 1 or more base cases that are easy to solve

```
def f(n):  
    return f(n-1)
```

```
def f(n):  
    if n==1:  
        return  
    return f(n-1)
```

Iterative Algorithms So Far

- looping constructs (`while` and `for` loops) lead to **iterative** algorithms
- can capture computation in a set of **state variables** that update on each iteration through loop

Multiplication - Iterative Solution

- “multiply $a * b$ ” is equivalent to “add a to itself b times”

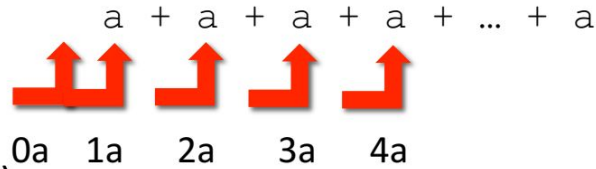
- capture **state** by

- an **iteration** number (i) starts at b

- $i \leftarrow i - 1$ and stop when 0

- a current **value of computation** (result)

- $result \leftarrow result + a$



```
def mult_iter(a, b):
```

```
    result = 0
```

```
    while b > 0:
```

```
        result += a
```

```
        b -= 1
```

```
    return result
```

iteration
current value of computation,
a running sum
current value of iteration variable

Multiplication - Recursive Solution

■ recursive step

- think how to reduce problem to a **simpler/smaller version** of same problem

$$\begin{aligned} a * b &= \underbrace{a + a + a + a + \dots + a}_{b \text{ times}} \\ &= a + \underbrace{a + a + a + \dots + a}_{b-1 \text{ times}} \\ &= a + \boxed{a * (b-1)} \end{aligned}$$

recursive reduction

■ base case

- keep reducing problem until reach a simple case that can be **solved directly**
- when $b = 1$, $a * b = a$

```
def mult(a, b):  
    if b == 1:  
        return a  
    else:  
        return a + mult(a, b-1)
```

base case

recursive step

Factorial - Recursive Solution

$$n! = n * (n-1) * (n-2) * (n-3) * \dots * 1$$

- for what n do we know the factorial?

$n = 1$ $\rightarrow$ `if n == 1:`
 `return 1` *base case*

- how to reduce problem? Rewrite in terms of something simpler to reach base case

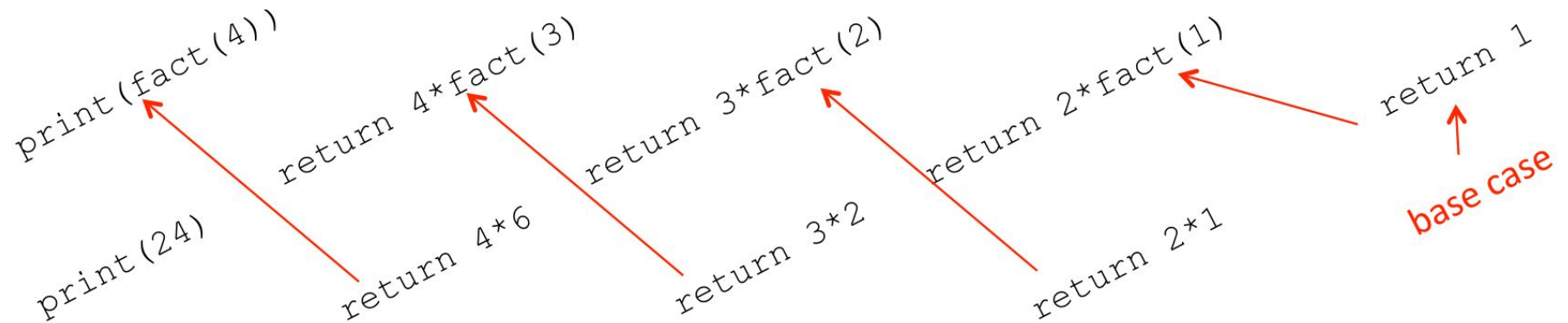
$n*(n-1)!$ $\rightarrow$ `else:`
 `return n*factorial(n-1)`

recursive step

Factorial - Recursive Solution

```
def fact(n):  
    if n == 1:  
        return 1  
    else:  
        return n*fact(n-1)
```

```
print(fact(4))
```



Multiple Recursive Calls

■ Fibonacci numbers

- Leonardo of Pisa (aka Fibonacci) modeled the following challenge
 - Newborn pair of rabbits (one female, one male) are put in a pen
 - Rabbits mate at age of one month
 - Rabbits have a one month gestation period
 - Assume rabbits never die, that female always produces one new pair (one male, one female) every month from its second month on.
 - How many female rabbits are there at the end of one year?

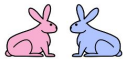
Multiple Recursive Calls



Demo courtesy of Prof. Denny Freeman and Adam Hartz

Month	Count
1	1
2	1

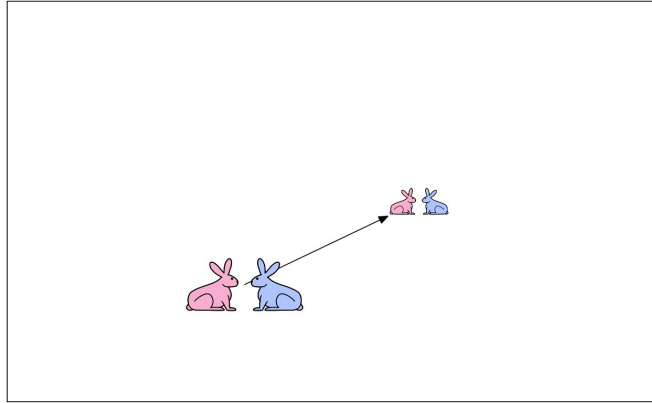
Multiple Recursive Calls



Demo courtesy of Prof. Denny Freeman and Adam Hartz

Month	Count
1	1
2	1

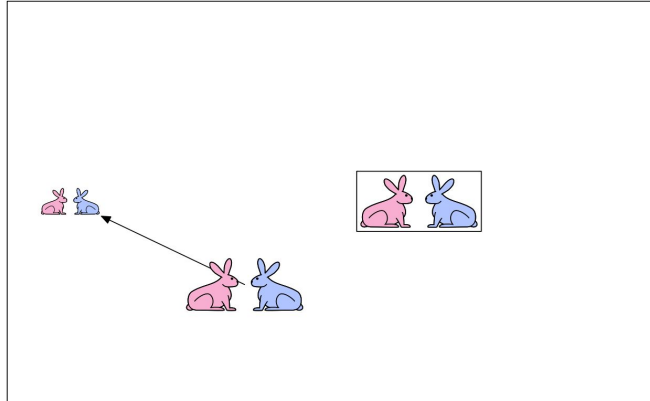
Multiple Recursive Calls



Demo courtesy of Prof. Denny Freeman and Adam Hartz

Month	Count
1	1
2	1
3	2

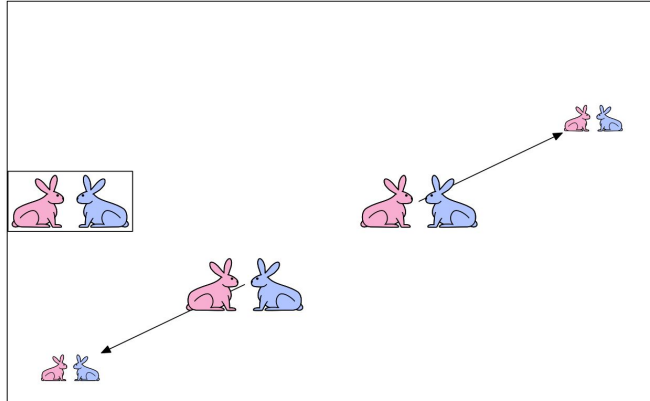
Multiple Recursive Calls



Demo courtesy of Prof. Denny Freeman and Adam Hartz

Month	Count
1	1
2	1
3	2
4	3

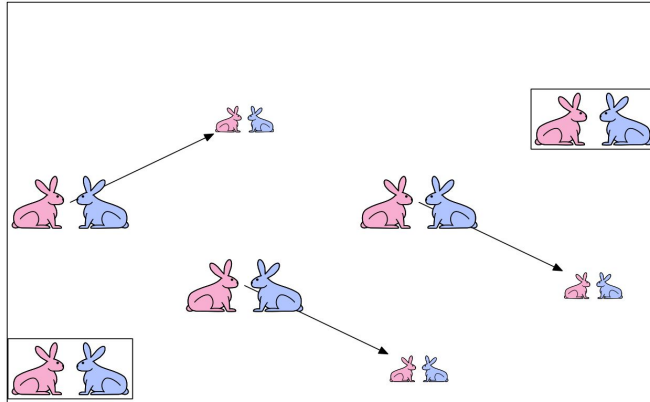
Multiple Recursive Calls



Demo courtesy of Prof. Denny Freeman and Adam Hartz

Month	Count
1	1
2	1
3	2
4	3
5	5

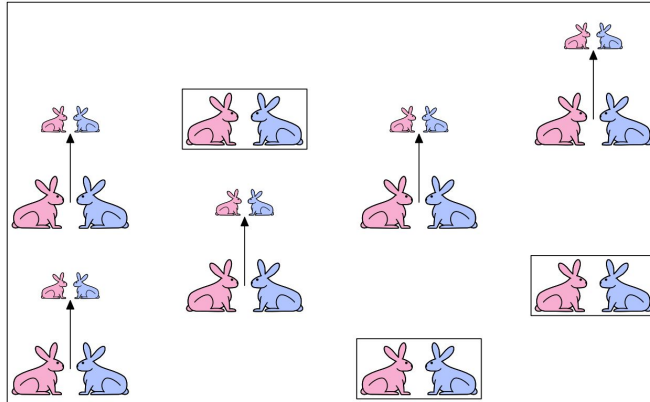
Multiple Recursive Calls



Demo courtesy of Prof. Denny Freeman and Adam Hartz

Month	Count
1	1
2	1
3	2
4	3
5	5
6	8

Multiple Recursive Calls

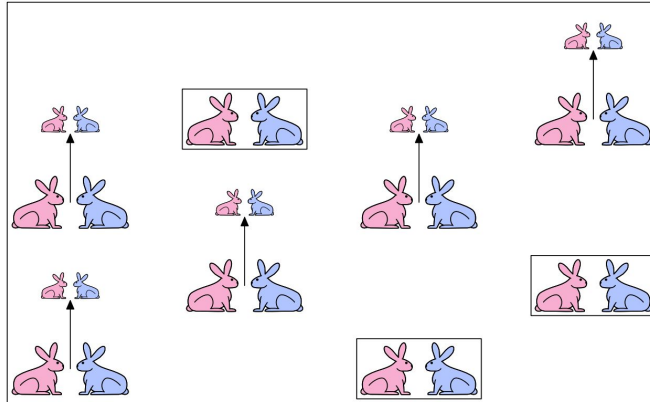


Demo courtesy of Prof. Denny Freeman and Adam Hartz

Month	Count
1	1
2	1
3	2
4	3
5	5
6	8
7	13

Fibonacci Numbers

$$f(n) = f(n-1) + f(n-2)$$



Demo courtesy of Prof. Denny Freeman and Adam Hartz

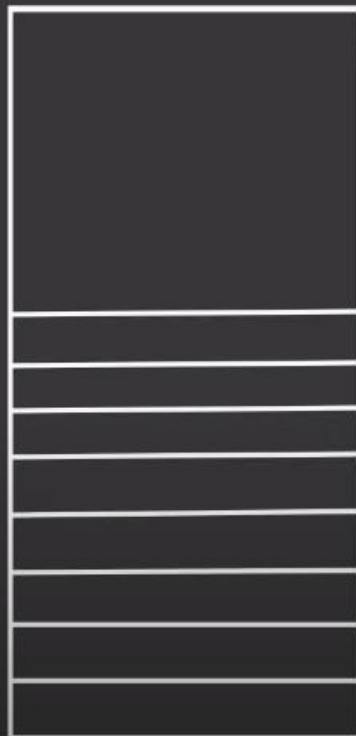
Month	Count
1	1
2	1
3	2
4	3
5	5
6	8
7	13

Fibonacci Numbers

```
def fib(x) :  
    """assumes x an int >= 0  
        returns Fibonacci of x"""  
    if x == 0 or x == 1:  
        return 1  
    else:  
        return fib(x-1) + fib(x-2)
```

```
fib(n)
{
  if(n== 1 || n== 2)
  {
    return 1;
  }
  return fib(n-1)+fib(n-2);
}
```

stack



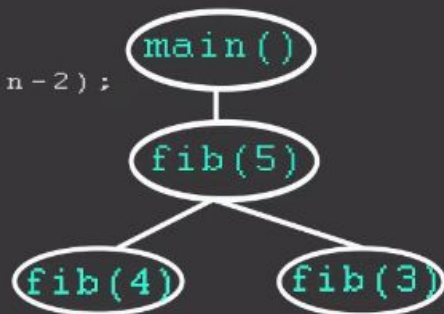
```
fib(n)
{
    if(n== 1 || n== 2)
    {
        return 1;
    }
    return fib(n-1)+fib(n-2);
}
```



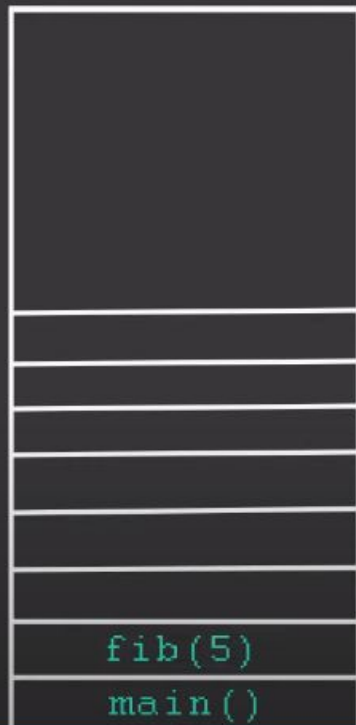
stack



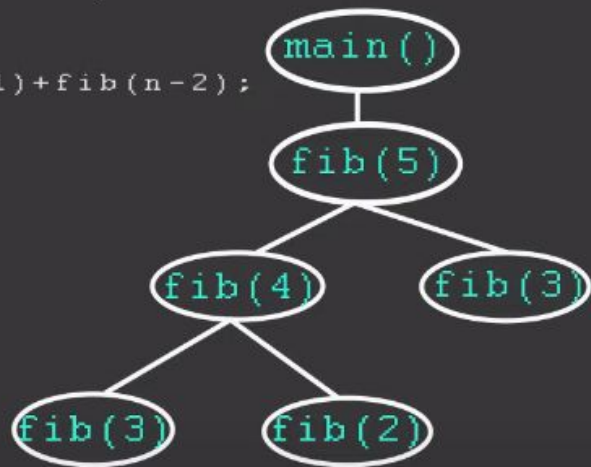
```
fib(n)
{
    if(n== 1 || n== 2)
    {
        return 1;
    }
    return fib(n-1)+fib(n-2);
}
```



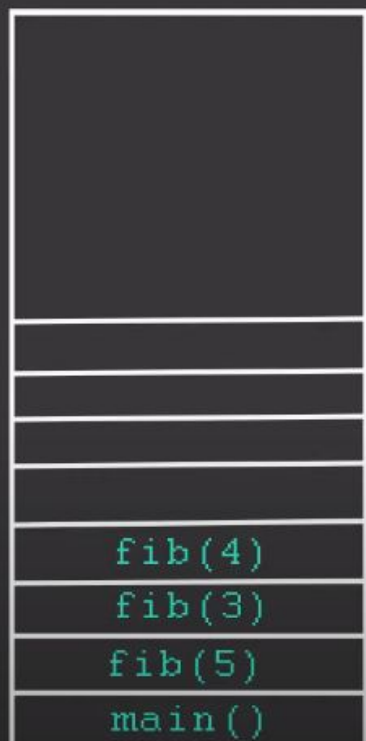
stack



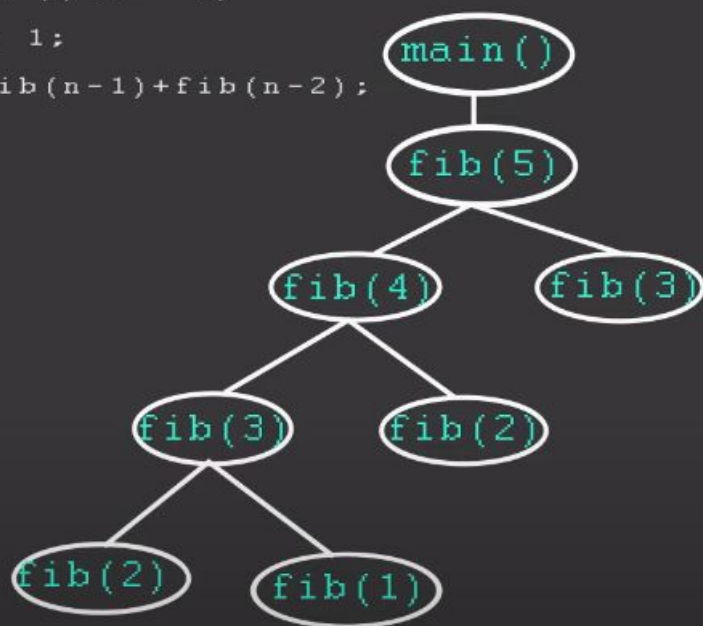
```
fib(n)
{
  if(n== 1 || n== 2)
  {
    return 1;
  }
  return fib(n-1)+fib(n-2);
}
```



stack



```
fib(n)
{
    if(n== 1 || n== 2)
    {
        return 1;
    }
    return fib(n-1)+fib(n-2);
}
```

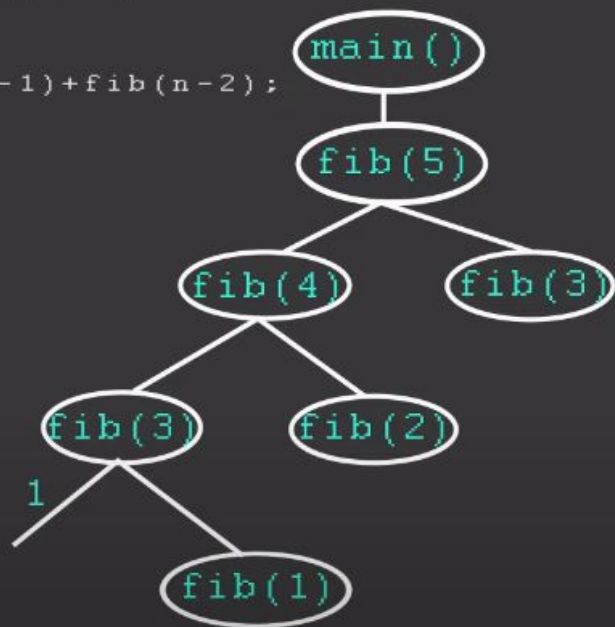


stack

fib(1)
fib(3)
fib(2)
fib(4)
fib(3)
fib(5)
main()



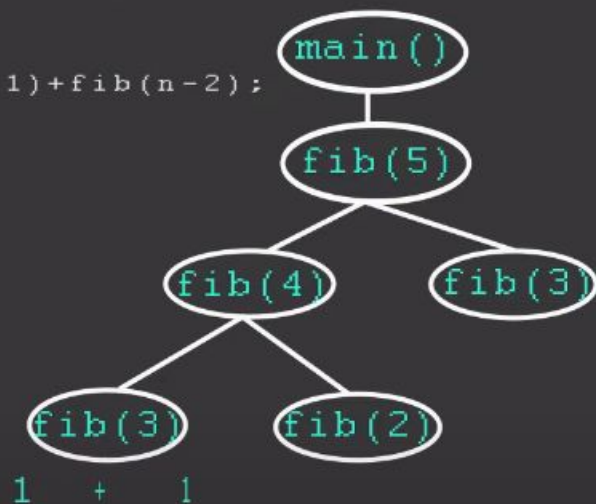
```
fib(n)
{
    if(n== 1 || n== 2)
    {
        return 1;
    }
    return fib(n-1)+fib(n-2);
}
```



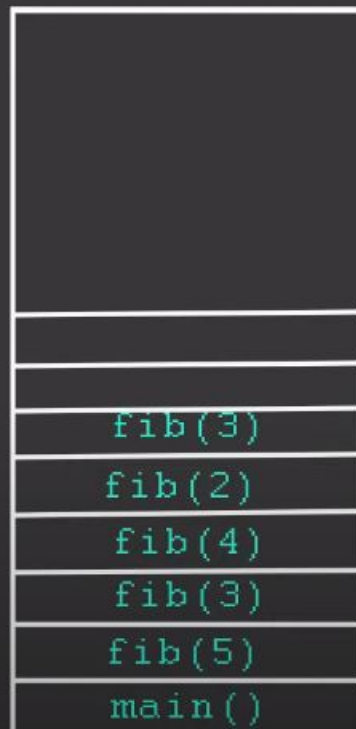
stack



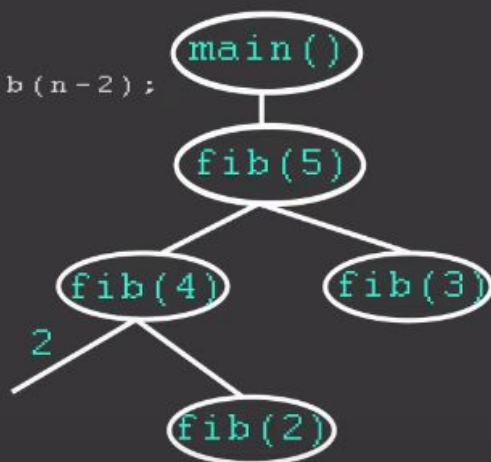

```
fib(n)
{
    if(n== 1 || n== 2)
    {
        return 1;
    }
    return fib(n-1)+fib(n-2);
}
```



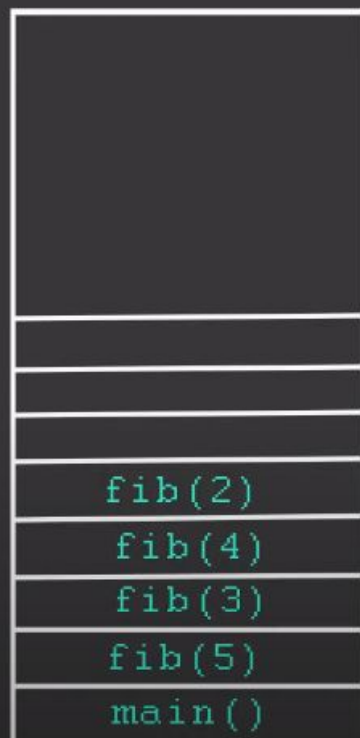
stack



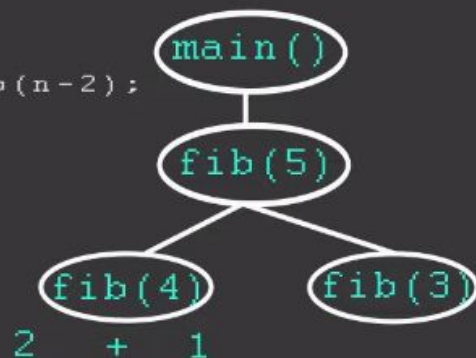
```
fib(n)
{
  if(n== 1 || n== 2)
  {
    return 1;
  }
  return fib(n-1)+fib(n-2);
}
```



stack



```
fib(n)
{
    if(n== 1 || n== 2)
    {
        return 1;
    }
    return fib(n-1)+fib(n-2);
}
```



stack

fib(4)

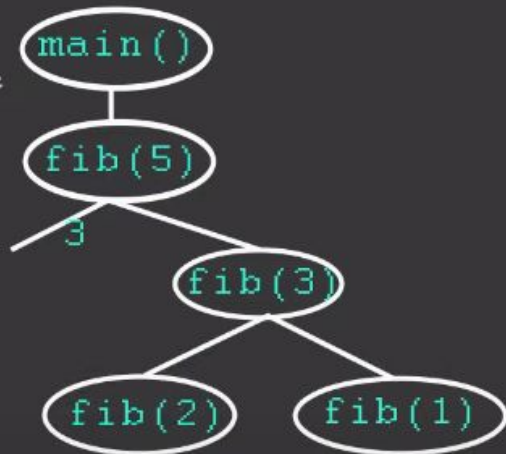
fib(3)

fib(5)

main()



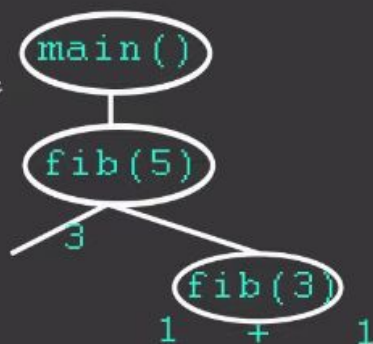
```
fib(n)
{
    if(n== 1 || n== 2)
    {
        return 1;
    }
    return fib(n-1)+fib(n-2);
}
```



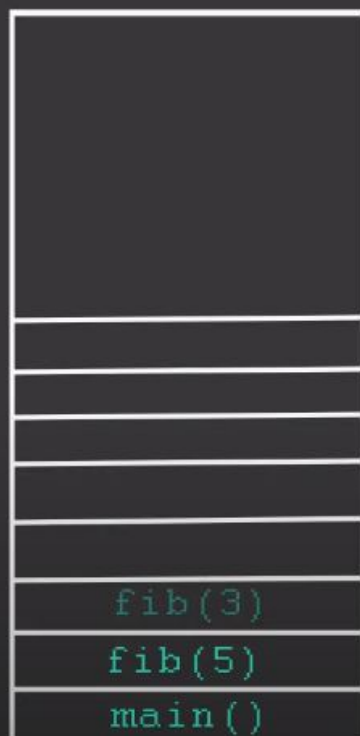
stack



```
fib(n)
{
  if(n== 1 || n== 2)
  {
    return 1;
  }
  return fib(n-1)+fib(n-2);
}
```



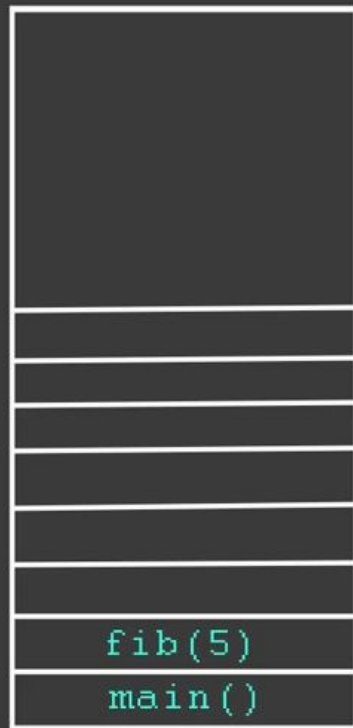
stack



```
fib(n)
{
    if(n== 1 || n== 2)
    {
        return 1;
    }
    return fib(n-1)+fib(n-2);
}
```



stack



```
fib(n)
{
    if(n== 1 || n== 2)
    {
        return 1;
    }
    return fib(n-1)+fib(n-2);
}
```

main()

5

stack

main()



Iteration Vs Recursion

```
def factorial_iter(n):  
    prod = 1  
    for i in range(1, n+1):  
        prod *= i  
    return prod  
  
def factorial(n):  
    if n == 1:  
        return 1  
    else:  
        return n*factorial(n-1)
```

- recursion may be simpler, more intuitive
- recursion may be efficient from programmer POV
- recursion may not be efficient from computer POV

Stack overflow
error!

Problem - Sum of Elements of Linked List

Let's now look at an example of structural recursion, one that uses a recursive data structure.

We want to compute the sum of the list of numbers. Linked list = 3-> 8-> 2-> 1-> 13-> 4->None

Problem - Sum of Elements of Linked List

```
def iterativeSum(head):  
    sum=0  
    temp=head  
    while temp!=None:  
        sum+=temp.element  
        temp=temp.next  
    return sum
```

Problem - Sum of Elements of Linked List

But a linked list is a recursive data structure. Hence, by thinking recursively, we note the following:

1. **Recursive Case:** The sum of the numbers in a list is nothing but the sum of the first number plus the sum of the rest of the list. The problem of summing the rest of the list is the same problem as the original, except that it's smaller by one element! Ah, recursion at play.
2. **Base Case:** The shortest list has a single number, which has the sum equal to the number itself. Now we have a base case as well. Note that if we allow our list of numbers to be empty, then the base case will need to be adjusted as well: the sum of an empty list is simply 0.

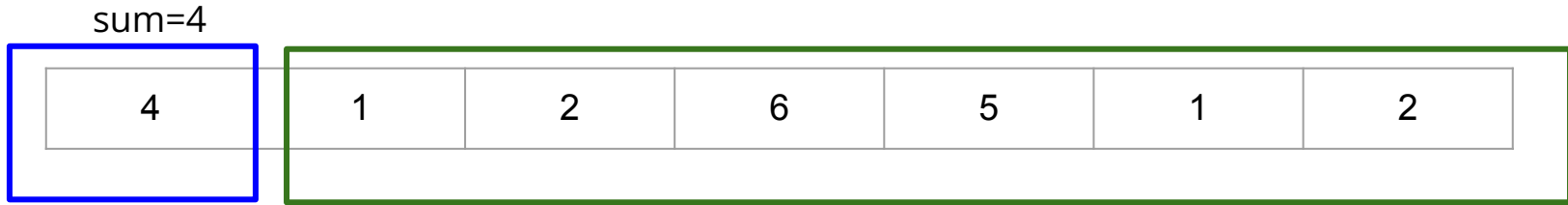
Problem - Sum of Elements of Linked List

4	1	2	6	5	1	2
---	---	---	---	---	---	---

Find the sum of above list

Problem - Sum of Elements of Linked List

Recursive approach:

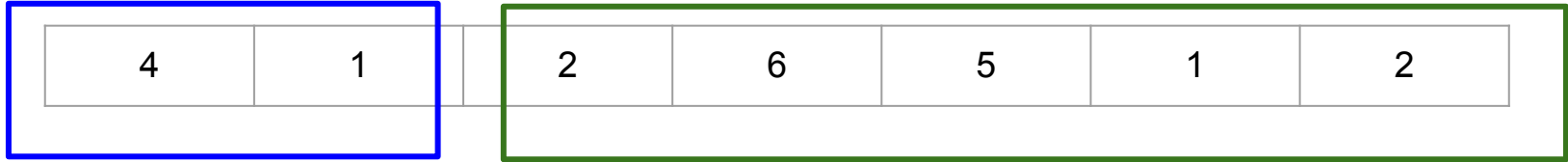


Find the sum of above list

Problem - Sum of Elements of Linked List

Recursive approach:

sum=5

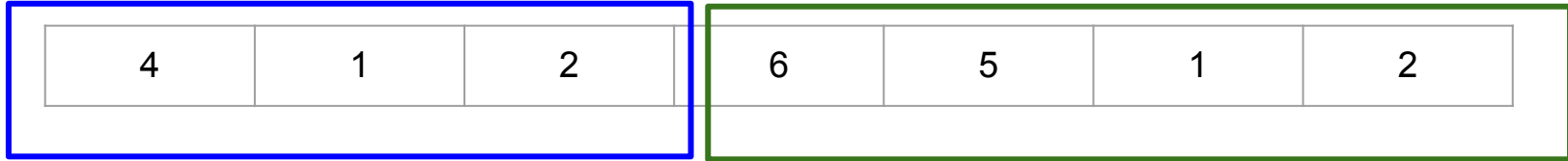


Find the sum of above list

Problem - Sum of Elements of Linked List

Recursive approach:

sum=7



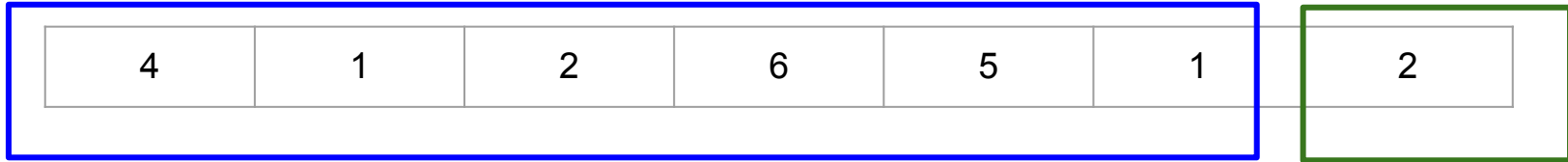
Find the sum of above list

Problem - Sum of Elements of Linked List

Recursive approach:

sum=19

Base case



Find the sum of above list

Problem - Sum of Elements of Linked List

Note the key difference between the iterative and recursive approaches:

for each number in the list

VS

the rest of the list

Problem - Sum of Elements of Linked List

Recursive Definition

```
sum = {
    | k                                k is the only element
    | k + sum(n.next)                 otherwise
}
```

Problem - Sum of Elements of Linked List

```
def sum(head) :  
    if head.next is None:  
        return head.data  
    else:  
        return head.data + sum(head.next)
```

Problem - Length of A String

```
len(s) =  $\begin{cases} 0 & \text{if string s is empty} \\ 1 + \text{len}(\text{rest}) & \text{otherwise} \end{cases}$ 
```

Problem - Length of A String

```
def strLength(str):  
    if len(str)==0: #length of the string is zero that means it is empty string  
        return 0    #Base case  
    else:  
        return 1+strLength(str[1: ])    #Recursive part
```

Problem - Length of a Linked List

```
len(l) =  $\begin{cases} 0 & \text{if } l \text{ is an empty list} \\ 1 + \text{len}(\text{rest}) & \text{otherwise} \end{cases}$ 
```

Problem - Length of a Linked List

```
def listLength(head):  
    if head==None: #empty linked list  
        return 0 #base case  
    else:  
        return 1+listLength(head.next) #Recursive part
```

Problem - Sequential Search in a Linked List

How would you find something in a linked list? Well, look at the first node and check if the key is in that node. If so, done. Otherwise, check the rest of the linked list for the given key. If you search an empty list for any key, the answer is false, so that's our base case.

```
contains(l, k) =  $\begin{cases} \text{false} & \text{if list } l \text{ is empty} \\ \text{true} & \text{if } l.\text{item} = k \\ \text{contains}(l.\text{next}, k) & \text{otherwise} \end{cases}$ 
```


Problem - Sequential Search in a Linked List

```
def contains(head, key):  
    if head == None:  
        return False    #base case  
    elif head.element == key:  
        return True     #base case  
    else:  
        return contains(head.next, key)    #recursive part
```

Problem - Sequential Search in an Array

Using array slicing:

```
def contains(arr,key):  
    if len(arr)==0:  
        return False    #base case  
    elif arr[0]==key:  
        return True     #base case  
    else:  
        return contains(arr[1: ],key)    #recursive part
```

Problem - Sequential Search in an Array

What if you need to implement this in a programming language that doesn't support slicing?

We can maintain a left index, along with a reference to the array, that is used to indicate where the beginning of the array is.

Initially,

$\text{left} = 0$, meaning that the array begins at the expected index of 0.

Eventually, a value of $\text{left} = \text{len}(\text{array}) - 1$ means that the rest of the array is simply the last element, and then

$\text{left} = \text{len}(\text{array})$ means that it's an 0-sized array.

Problem - Sequential Search in an Array

```
contains(a, l, k) =  $\begin{cases} \text{false} & \text{if } l \geq a.\text{length} \\ \text{true} & \text{if } a[l] = k \\ \text{return contains}(a, l+1, k) & \text{otherwise} \end{cases}$ 
```

Problem - Sequential Search in an Array

```
def contains(arr, left, key):  
    if left >= len(arr):  
        return False    #base case  
    elif arr[left] == key:  
        return True     #base case  
    else:  
        return contains(arr, left+1, key)    #recursive part
```

Problem - Binary Search

```
if the array is empty (if  $l > r$  that is):  
    return false  
else:  
    Find the position of the middle element:  $mid = (l + r) / 2$   
    If  $key == data[mid]$ , then return true  
    If  $key > data[mid]$ , the search the right half  $data[mid+1..r]$   
    If  $key < data[mid]$ , the search the left half  $data[l..mid-1]$ 
```

Problem - Binary Search

```
contains(a,l,r,k) = -
                    | false                if l > r
                    | true                 if k = a[mid]
                    |
                    | contains(a,mid+1,r,k) if k > a[mid]
                    | contains(a,l,mid-1,k) if k < a[mid]
                    -
```

Problem - Binary Search

```
def contains(arr, left, right, key):  
    if left > right:  
        return False    #base case  
    else:  
        mid=(left+right)//2  
        if key==arr[mid]:  
            return True    #base case  
        elif key > arr[mid]:  
            return contains(arr, mid+1, right, key) #recursive part  
        else:  
            return contains(arr, left, mid-1, key) #recursive part
```


Problem - Finding the Max in Sequence

Formulating this recursively: the maximum key in a sequence is the larger of the following two:

1. the 1st key in the sequence
2. the maximum key in the rest of the sequence

Problem - Finding the Max in Sequence

```
def maximum(a,b):  
    return a if a>=b else b  
  
def findMax(head):  
    if head.next==None:  
        return head.element    #base case  
    else:  
        maxRest=findMax(head.next)    #recursive part  
        return maximum(head.element,maxRest)
```

Problem - Finding the Max in an Array

```
def maximum(a,b):  
    return a if a>=b else b  
  
def findMax(arr, left):  
    if left == len(arr)-1:  
        return arr[left]    #base case  
    else:  
        maxRest=findMax(arr, left+1)    #recursive part  
        return maximum(arr[left], maxRest)
```

Problem - Finding the Max in an Array (Binary Search)

```
def maximum(a,b):  
    return a if a>=b else b  
  
def findMax(arr, left, right):  
    if left == right:  
        return arr[left]    #base case  
    else:  
        mid = (left+right)//2  
        maxLeftHalf=findMax(arr, left, mid)    #recursive part  
        maxRightHalf=findMax(arr, mid+1, right) #recursive part  
        return maximum(maxLeftHalf, maxRightHalf)
```

Question - 01

Given a non-negative int n, return the sum of its digits recursively (no loops). Note that mod (%) by 10 yields the rightmost digit (126 % 10 is 6), while divide (/) by 10 removes the rightmost digit (126 / 10 is 12).

Example:

Input: sumDigits(126)

Output: 9

Input: sumDigits(49)

Output: 13

Input: sumDigits(12)

Output: 3

```
def sumDigits(n):  
    # Base case: if n is a single-digit number, return n  
    if n < 10:  
        return n  
    else:  
        # Recursive step: get the last digit (rightmost) using modulo (%) and add it to  
        the sum of digits of the rest of the number  
        return n % 10 + sumDigits(n // 10)
```

Test cases

```
print(sumDigits(126)) # Output: 9  
print(sumDigits(49))  # Output: 13  
print(sumDigits(12))  # Output: 3
```

Question - 02

We have bunnies standing in a line, numbered 1, 2, ... The odd bunnies (1, 3, ..) have the normal 2 ears. The even bunnies (2, 4, ..) we'll say have 3 ears, because they each have a raised foot. Recursively return the number of "ears" in the bunny line 1, 2, ... n (without loops or multiplication).

Example:

Input: bunnyEars2(0)

Output: 0

Input: bunnyEars2(1)

Output: 2

Input: bunnyEars2(2)

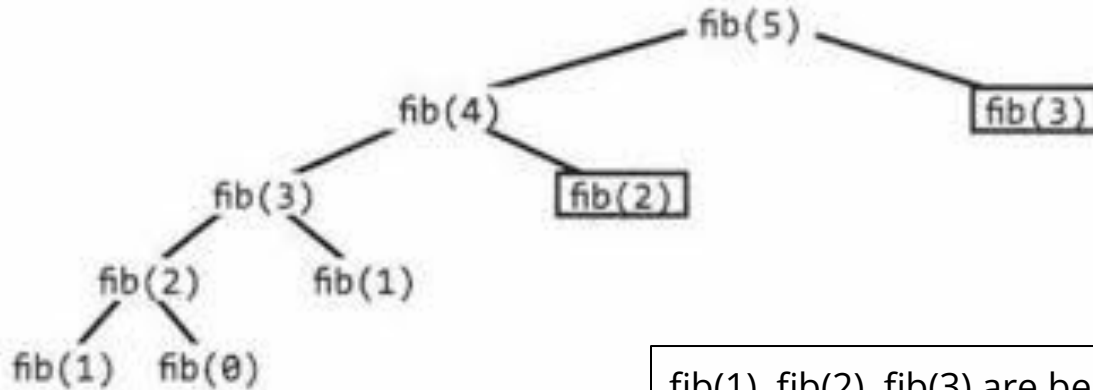
Output: 5

```
def bunnyEars2(n):  
    # Base case: if n is 0, return 0 (no bunnies, no ears)  
    if n == 0:  
        return 0  
    else:  
        # If n is odd, the bunny has 2 ears; otherwise, it has 3 ears (even bunnies have an extra raised foot)  
        if n % 2 == 1:  
            return 2 + bunnyEars2(n - 1)  
        else:  
            return 3 + bunnyEars2(n - 1)
```

Test cases

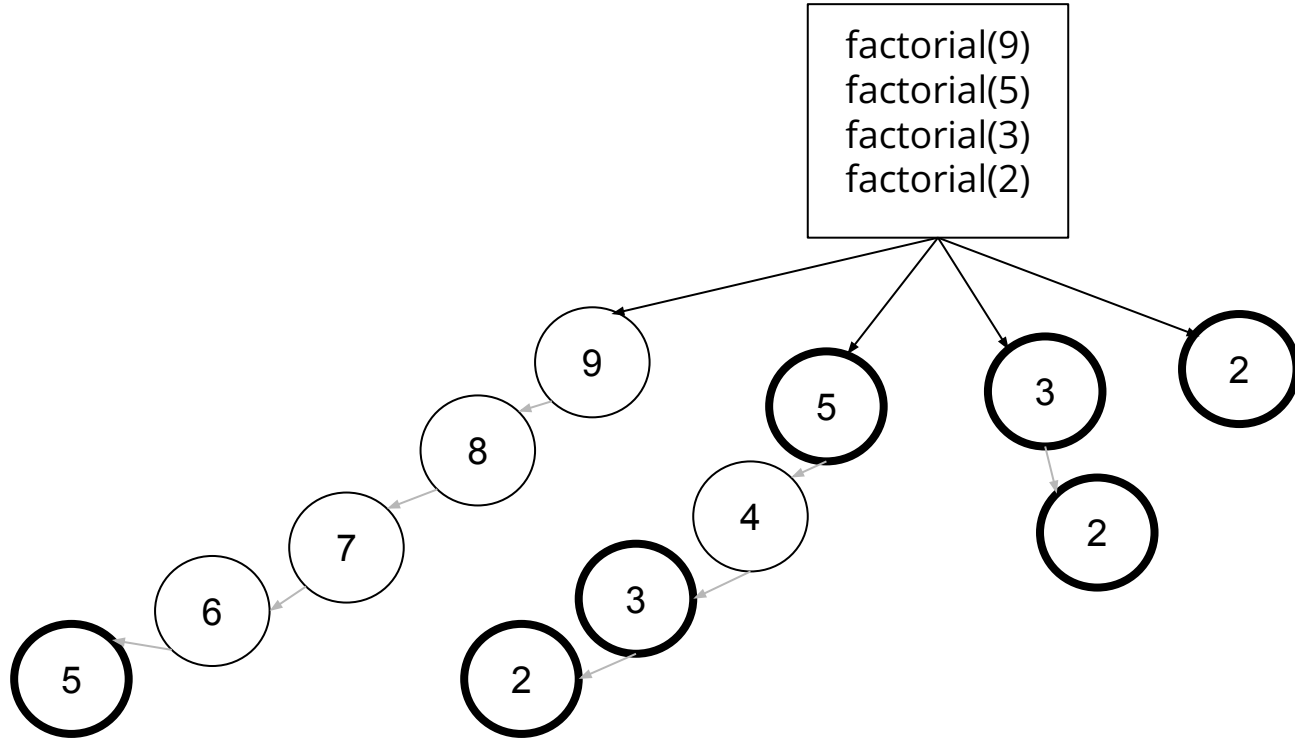
```
print(bunnyEars2(0)) # Output: 0  
print(bunnyEars2(1)) # Output: 2  
print(bunnyEars2(2)) # Output: 5
```


Optimization of Recursion: Memoization



fib(1), fib(2), fib(3) are being called multiple times. These are called overlapping subproblems.

Optimization of Recursion: Memoization



Memoizing the Fibonacci Problem

```
def fib(n):  
    # Create and initialize the table.  
    F = [-1]*(n+1)  
  
    #Now we can call M-Fib with this extra parameter "F".  
    return M_fib(n, F)  
  
def M_fib(n, F):  
    #The table "F" is being passed as a parameter  
  
    if n < 2:  
        return n    #base case  
  
    else:  
        #Compute (and save) if it's not already computed.  
        if F[n] == -1:  
            F[n] = M_fib(n-1, F) + M_fib(n-2, F)    #recursive part  
  
        #Now just return the computed (and saved) value.  
        return F[n]
```